

Ejercicio 8: Planificación Óptima de Repostajes (Algoritmo Voraz)

Este documento presenta el diseño y la justificación formal del algoritmo Voraz (*Greedy*) óptimo para calcular el número mínimo de paradas necesarias en el viaje del profesor Allilev desde Malanquilla hasta Zaragoza.

1. Estrategia Algorítmica (Propiedad de la Elección Voraz)

El problema consiste en viajar una distancia total a lo largo de una línea recta (la carretera comarcal 1313) donde disponemos de un coche con autonomía máxima de n kilómetros.

Elección Voraz

Dado que cada repostaje llena al 100% el depósito de combustible del coche (reseteando la autonomía disponible a n kilómetros), no existe ninguna ventaja en repostar antes de lo estrictamente necesario.

La estrategia óptima consiste en **estirar al máximo cada depósito**, circulando el mayor número de kilómetros posibles sin quedarse sin combustible. El profesor debe detenerse a repostar en la **última estación de servicio alcanzable** antes de que su depósito se agote por completo en la carretera.

- **Paso Voraz:** En cada etapa del viaje, si nos encontramos en la estación i (o en el origen), seleccionamos como siguiente parada la estación de servicio j que esté más alejada a la derecha de nuestra posición actual, de modo que la distancia entre ambas sea menor o igual que la autonomía n :

$$d(i, j) \leq n \quad \text{y} \quad d(i, j + 1) > n$$

2. Diseño del Algoritmo en Pseudocódigo

Parámetros del Problema

- $E = [e_0, e_1, \dots, e_m]$: Un vector ordenado con las posiciones (kilómetros acumulados) de las estaciones de servicio, donde:
 - $e_0 = 0$: Posición de salida (Malanquilla).
 - e_m : Posición de destino (Zaragoza).
- n : Autonomía máxima del automóvil con el depósito lleno.

Pseudocódigo

```
Funcion PlanificarRepostajesAllilev(E, m, n)
```

```
Inicio
```

```
    estaciones_parar <- Lista vacía // Almacenará los índices de las estaciones d
    posicion_actual <- 0             // Índice de la estación actual (empezamos en
```

```

i <- 1 // Índice para explorar las siguientes estaci

Mientras posicion_actual < m Hacer
    ultima_alcanzable <- posicion_actual

    // Avanzar el puntero mientras las estaciones sean físicamente alcanzable
    Mientras i <= m y (E[i] - E[posicion_actual]) <= n Hacer
        ultima_alcanzable <- i
        i <- i + 1
    FinMientras

    // 1. Control de Viabilidad: Si no es posible avanzar ni una estación
    Si ultima_alcanzable = posicion_actual Entonces
        Escribir("Error: No es posible completar el viaje. Autonomía insufici
        Retornar Lista vacía
    FinSi

    // 2. Comprobar si hemos alcanzado el destino final (Zaragoza)
    Si ultima_alcanzable = m Entonces
        posicion_actual <- m
    Sino
        // 3. Si no hemos llegado, paramos en la última alcanzable para repos
        Añadir ultima_alcanzable a estaciones_parar
        posicion_actual <- ultima_alcanzable
    FinSi
FinMientras

Retornar estaciones_parar
FinFuncion

```

3. Análisis de Complejidad

- **Complejidad Temporal:** $\Theta(m)$, donde m es el número de estaciones de servicio en el mapa. A pesar de los dos bucles `Mientras` anidados, el puntero de exploración i se incrementa de forma estrictamente monótona de 1 a m . Ninguna estación se analiza más de dos veces (una vez para validar su distancia y otra para establecer el nuevo origen). Por tanto, el número total de operaciones es lineal respecto a m .
- **Complejidad Espacial:** $O(m)$ en el peor caso para almacenar los índices de las paradas seleccionadas en la lista de salida.

4. Justificación de la Corrección (Demostración de Optimalidad)

Para demostrar que la estrategia voraz del profesor Allilev minimiza el número de paradas, utilizaremos una demostración por inducción basada en el argumento de "El algoritmo voraz siempre va por delante" (*Greedy stays ahead*).

Sean:

- $G = (g_1, g_2, \dots, g_k)$ la secuencia de índices de las estaciones elegidas por nuestro algoritmo voraz.
- $O = (o_1, o_2, \dots, o_p)$ la secuencia de índices de las estaciones elegidas por una solución óptima cualquiera para completar el viaje.

Enunciado de la Hipótesis Inductiva

Para cada parada j común ($1 \leq j \leq \min(k, p)$), la posición física alcanzada en la comarcal 1313 por el algoritmo voraz tras realizar la j -ésima parada está al menos tan lejos o más que la alcanzada por la solución óptima:

$$E[g_j] \geq E[o_j]$$

Demostración de la Hipótesis

1. Caso Base ($j = 1$):

- En la primera parada, saliendo de Malanquilla ($E[e_0] = 0$), el algoritmo voraz elige la estación g_1 más lejana posible que cumpla $E[g_1] \leq n$.
- Cualquier solución válida O debe realizar su primera parada a una distancia menor o igual que n para no quedarse sin combustible. Por definición de máximo de nuestro paso voraz, se cumple:

$$E[o_1] \leq E[g_1] \quad (\text{Se cumple el caso base})$$

2. Paso Inductivo:

- Supongamos que la hipótesis es cierta para la parada $j - 1$, es decir:

$$E[g_{j-1}] \geq E[o_{j-1}]$$

- Queremos probar que la relación se mantiene para la parada j .
- Desde la posición óptima o_{j-1} , la solución óptima avanza hasta su siguiente parada o_j . Para que esta transición sea viable, debe cumplirse la restricción de autonomía:

$$E[o_j] - E[o_{j-1}] \leq n \implies E[o_j] \leq E[o_{j-1}] + n$$

- Aplicando nuestra hipótesis de inducción:

$$E[o_j] \leq E[o_{j-1}] + n \leq E[g_{j-1}] + n$$

- Esto significa que la estación o_j de la solución óptima se encuentra dentro del rango de alcance máximo factible si repostáramos en la parada voraz anterior g_{j-1} (ya que $E[o_j] - E[g_{j-1}] \leq n$).
- Como el algoritmo voraz elige por diseño la estación g_j más lejana posible dentro de su rango de alcance, obligatoriamente se cumplirá que:

$$E[g_j] \geq E[o_j]$$

Conclusión de Optimalidad

Dado que $E[g_j] \geq E[o_j]$ para todo paso j , en el momento en que la solución óptima alcanza el destino (Zaragoza) en su parada p ($o_p = m$), se cumplirá que:

$$E[g_p] \geq E[o_p] = E[m]$$

Esto implica que el algoritmo voraz habrá alcanzado Zaragoza en p o en menos paradas ($k \leq p$). Dado que O es una solución óptima y no existe ninguna que pueda requerir estrictamente menos paradas que el óptimo, se concluye que $k = p$, demostrando que el algoritmo voraz es globalmente óptimo.